Privacy Engineering

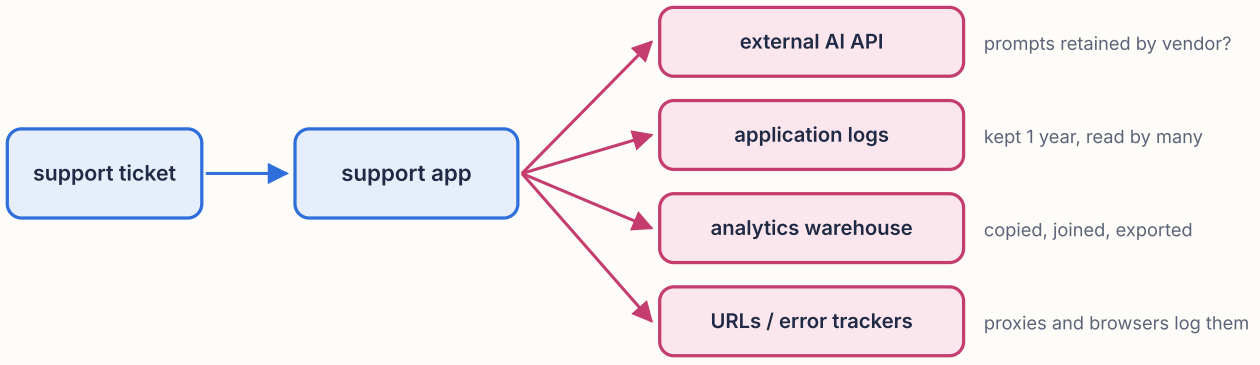
design so personal data can't leak by accident

Watch the 4-min video

In one line: collect and keep only the personal data you need; find and classify it; reduce exposure with masking, tokenisation and pseudonymisation; keep it out of logs, URLs and analytics; enforce consent and deletion across every copy; and review every third-party flow, including AI APIs, like any other data flow.

1 Nobody intended it

A support assistant built on an LLM is a hit. Then an audit finds customer names, phone numbers and card details from tickets were sent to an external AI service, written to application logs, and copied into the analytics warehouse. No one decided to do this; each step was a reasonable default. **Privacy engineering** makes the safe path the default, so leaks don't depend on every developer remembering every rule.



Unintended copies: each one is a place to secure, retain, delete, and report in a breach.

Figure 1. One ticket, four unintended copies. Each copy needs its own access control, retention and deletion, and each is in scope for breach reporting.

2 Privacy by design

GDPR Article 25 calls it **data protection by design and by default**; the principles (Article 5) translate directly into engineering rules:

- **Data minimisation**: collect only fields with a stated purpose. Don't store date of birth if you only need "over 18".
- **Purpose limitation**: data collected for support isn't automatically available for marketing models.
- **Storage limitation**: every table, bucket and log stream has a retention period and a deletion job.
- **Least access**: people and services see only what their task needs.

Every new feature answers three questions in its design review: **what personal data** does it use, **why**, and **where does it flow** (including vendors)? For high-risk processing, the formal version is a Data Protection Impact Assessment (DPIA).

3 Find and classify

You can't protect what you can't find. **PII discovery** scans databases, files and logs using regular expressions (emails, phone formats), checksums (Luhn for card numbers, check digits for national IDs), dictionaries and ML named-entity recognition (names, addresses). Tools include Microsoft Presidio (open source), Amazon Macie, Google Cloud DLP / Sensitive Data Protection and Microsoft Purview. **Classification** then tags each column or field, and those tags drive the controls automatically (lesson 68).



Figure 2. A typical four-level scheme. The tag on a column decides access, masking, encryption and retention, not each team's judgement.

4 Reduce exposure

Technique	What it does	Reversible?	Example use
Masking	hides part of a value for display	no (view only)	phone `•••• ••• 678`; card shows last 4
Tokenisation	replaces a value with a random token; a secured vault maps it back	yes, only via the vault	card numbers: most systems store `tok_8f3…`, shrinking PCI DSS scope
Pseudonymisation	consistent alias for an identifier (keyed hash, e.g. HMAC-SHA-256)	only with the key or mapping	analysts count distinct users without knowing who
Encryption	ciphertext; key holders can decrypt	yes, with the key	field-level encryption of national IDs
Anonymisation / aggregation	removes the link to individuals	no (if done properly)	published statistics

Two traps. A **plain hash** of an email or phone number is not anonymous: the input space is small, so anyone can hash a list of candidates and match. Use a **keyed** hash and protect the key. And under GDPR, **pseudonymised data is still personal data**; only truly anonymised data falls outside it. Treat pseudonymised datasets as sensitive, just less exposed.

5 Differential privacy, consent and deletion

Differential privacy adds calibrated random noise to aggregate results so that the output is almost the same whether or not any one person is in the data. The parameter ε (epsilon) is the privacy budget: smaller ε means more noise and stronger privacy, and every query spends some of the budget. The US Census Bureau used it for the 2020 census; Apple and Google use variants for telemetry. It suits statistics over many people, not individual-level lookups.

Consent management records what each user agreed to (purpose, timestamp, version of the wording) and makes it checkable at the point of use: the marketing pipeline filters on `consent.marketing = true`, and withdrawal propagates within a defined time.

Data subject requests (access, export, deletion) must reach **every copy**: primary DB, search, caches, warehouse, backups, logs and vendors. A data map and lineage (lesson 68) are what make that possible. For data you can't easily rewrite (backups, event logs), **crypto-shredding** helps: encrypt each user's data with a per-user key and delete the key to make every copy unreadable.

6 Where PII leaks most

- **Logs**: request bodies, SQL with parameters, exception messages. Scrub at the source with logging filters and structured logs with an allow-list of fields; set short retention.
- **URLs**: `/reset?email=…` ends up in proxy logs, browser history, analytics and Referer headers. Put sensitive values in request bodies or headers.
- **Analytics and ML**: give pseudonymised, minimised datasets by default; raw access is an exception with approval.
- **Third parties**, especially AI APIs: what is sent, is it retained or used for training, where is it processed (data residency, lesson 46), and is there a data processing agreement?
- **Lower environments**: production copies in dev and test. Use synthetic or masked data.

7 Masking PII before an LLM

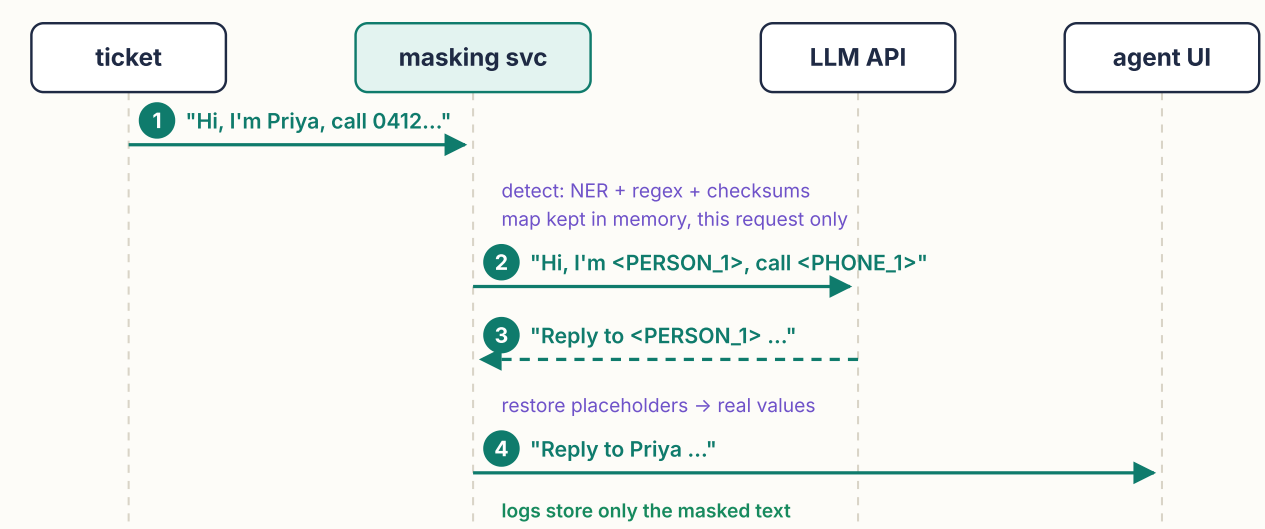


Figure 3. Reversible masking around an LLM call. The model only ever sees placeholders; the mapping never leaves the request's memory.

WORKED EXAMPLE: MASKING PII BEFORE SENDING TEXT TO AN LLM

Before a ticket goes to the model, a PII detection service (for example Presidio's analyzer) finds names, emails, phone numbers and card numbers, combining NER, regular expressions and the Luhn check, and replaces each with a **consistent placeholder**: `<PERSON_1>`, `<PHONE_1>`. Consistency matters: the same person is `<PERSON_1>` everywhere, so the model can still reason about who said what. The placeholder-to-value map is held in memory for this request only. The model's answer uses the placeholders; the service substitutes the real values before showing the draft reply to the support agent. Card numbers are never restored at all: the reply only needs the last four digits. Logs and traces store only the masked text, and the vendor contract specifies no training on inputs and limited retention. Detection is not perfect, so this is one layer of defence, alongside minimising what goes into the prompt in the first place.

COMMON MISTAKES

- Hashing emails without a key and calling the result anonymous.
- Deleting a user from the main DB but not from search, warehouse, logs and vendors.
- Logging full request bodies "temporarily" in production.
- Treating an AI API as a library call instead of a data transfer to a third party.

8 Key terms

PII: information that identifies a person, directly or combined

Tokenisation: random token replaces a value; only a vault maps it back

Differential privacy: calibrated noise so individuals can't be inferred

Data minimisation: collect and keep only what a purpose needs

Pseudonymisation: consistent aliases; still personal data

Crypto-shredding: delete a per-user key to erase all their encrypted copies

9 Interview questions

Q1. How do you design a system to minimise PII exposure? BASIC

Collect and keep only what you need; discover and classify personal data; protect it with masking, tokenisation and pseudonymisation; scrub it from logs, URLs and analytics; enforce consent and deletion across every copy; and review third-party flows like AI APIs.

Q2. What is tokenisation? BASIC

Replacing a sensitive value with a random token that only a secure vault can map back, so most systems never hold the real value.

Q3. Masking vs pseudonymisation? BASIC

Masking hides part of a value for display and is one-way; pseudonymisation replaces an identifier with a consistent alias so records can still be linked and counted.

Q4. A user asks to be deleted. What has to happen? SCENARIO

Find every copy via the data map and lineage (DB, search, caches, warehouse, logs, vendors), delete or anonymise them within the legal deadline, and handle backups by expiry or crypto-shredding.

Q5. Analysts need user-level behaviour data. How do you give it safely? SCENARIO

Provide a pseudonymised dataset with keyed-hash user IDs, minimised columns and no direct identifiers, under access controls, with re-identification keys held separately.

Q6. Is SHA-256 of an email address anonymous? TRICKY

No: emails are guessable, so a dictionary of candidate emails can be hashed and matched; use a keyed hash and treat the result as personal data.

Q7. Why keep the placeholder map only in memory in the LLM example? TRICKY

Persisting it would create a new store linking placeholders to real PII, which then needs its own security, retention and deletion; per-request memory avoids the copy.

Next: SD-V-018 · Data Governance and Lineage backendroadmap.com